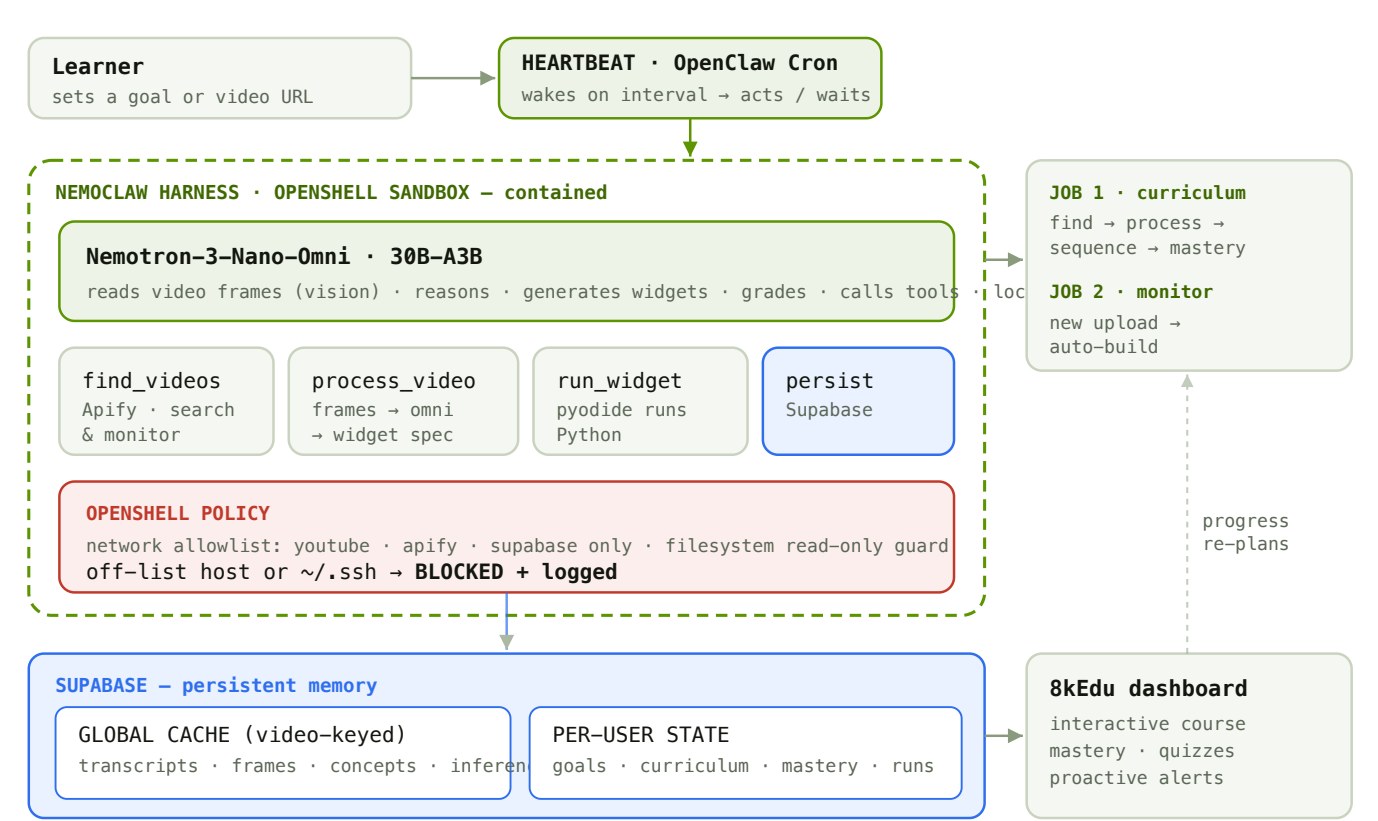


8kEdu

YouTube video → interactive learning dashboard

An autonomous learning agent: give it a goal, it builds and maintains your course on a heartbeat — finding lectures, turning each into live widgets, tracking mastery, watching for new uploads — with no step-by-step prompting.

System at a glance



Components

LAYER	WHAT IT IS
Agent harness	NemoClaw runs an OpenClaw agent inside an OpenShell sandbox; heartbeat via OpenClaw Cron jobs.
Brain	nemotron-3-nano-omni (30B-A3B, ~3B active) — one omni model does vision, reasoning, and tool-calling. Runs local (LM Studio / vLLM profile).
Tools	find_videos (Apify), process_video (yt-dlp + ffmpeg frames → omni → concept spec), run_widget (pyodide), persist (Supabase).
Containment	OpenShell policy: per-binary network egress allowlist (enforce / audit) + filesystem read-only/read-write + landlock.
Memory	Supabase Postgres, two tiers — global content cache (shared across users) + per-user learner state.
Surface	Vite + React dashboard: player, timeline, transcript, live widgets, notebooks, exports.

Data model — two tiers

Global cache (keyed by YouTube video id, shared by all learners): `videos` · `transcripts` · `frames` · `concepts` · `inference_cache` . A video is analyzed once; every learner reuses it, so marginal cost approaches zero. **Per-user state**: `goals` · `curriculum` · `mastery` · `monitored_channels` · `runs` — the agent's recoverable workspace and session history.

The heartbeat loop

HEARTBEAT (OpenClaw Cron) — wake, read state from Supabase:

```
JOB 1 · advance curriculum
  goal not met?
    gap in coverage → Apify: find a lecture for the weakest concept
    unprocessed video → process_video: frames → omni → widgets
    new material      → Nemotron: re-sequence course, update mastery
  else → idle till next tick

JOB 2 · monitor channels
  new upload since last cycle? → build its dashboard → alert learner

every cycle → write run log + state (recoverable)
quiz results → re-plan next cycle
```

Run locally

`uv sync` and `cd app && npm install`, then `npm run dev` (frontend, pre-baked demos work with zero model calls). To process a new video: `uv run ingest.py "<url>"` → `uv run analyze.py --backend mlx --video <id>` → `uv run serve.py --backend mlx`. Cloud backends are blocked unless `TACTILE_ALLOW_CLOUD=1`.